EXPERIMENT 04

Polynomial Regression and Bias-Variance Trade-off

Python practical program using the California Housing dataset

```
# Experiment 04: Polynomial Regression and Bias-Variance Trade-off

# 1. Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load dataset
data = fetch_california_housing()

df = pd.DataFrame(data.data, columns=data.feature_names)
df['MedHouseVal'] = data.target

# 3. Display first five records
print("First Five Records:")
print(df.head())

# 4. Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# 5. Select input and output
X = df[['MedInc']]
y = df['MedHouseVal']

# 6. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 7. Apply Polynomial Regression
degrees = [1, 2, 5]
results = []

for degree in degrees:

    model = make_pipeline(
        PolynomialFeatures(degree),
        LinearRegression()
    )

    model.fit(X_train, y_train)

    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    train_mse = mean_squared_error(y_train, train_pred)
    test_mse = mean_squared_error(y_test, test_pred)
    rmse = np.sqrt(test_mse)
    r2 = r2_score(y_test, test_pred)

    results.append([degree, train_mse, test_mse, rmse, r2])

# 8. Display comparison results
result = pd.DataFrame(results, columns=[
    'Degree', 'Train MSE', 'Test MSE', 'RMSE', 'R2 Score'
])

print("\nPolynomial Regression Results:")
print(result)

# 9. Plot Polynomial Regression
plt.figure(figsize=(7, 4))

plt.scatter(X_test['MedInc'], y_test,
```

```
            alpha=0.2, label='Actual Data')

# Create graph input with feature name
x_plot = pd.DataFrame({
    'MedInc': np.linspace(
        X['MedInc'].min(),
        X['MedInc'].max(),
        200
    )
})

# Draw curves for different degrees
for degree in degrees:

    model = make_pipeline(
        PolynomialFeatures(degree),
        LinearRegression()
    )

    model.fit(X_train, y_train)
    y_plot = model.predict(x_plot)

    plt.plot(x_plot['MedInc'], y_plot,
             label=f'Degree {degree}')

# Graph labels
plt.xlabel("Median Income")
plt.ylabel("House Value")
plt.title("Polynomial Regression")
plt.legend()
plt.show()

print("\nPolynomial Regression Completed Successfully!")
```